What is Git? (Interview Answer)

Definition

Git is a **distributed version control system (DVCS)** used to track changes in source code, manage different versions of files, and enable multiple developers to collaborate on the same project without overwriting each other's work.

Interview Answer (2 Minutes)

"Git is a distributed version control system used to track changes in source code and collaborate with other developers. It allows multiple team members to work on the same project simultaneously using branches. Git maintains a complete history of changes, making it easy to revert to previous versions if needed. In DevOps, Git is commonly used to store application code, infrastructure-as-code files, and CI/CD pipeline configurations. Platforms like GitHub, GitLab, and Bitbucket provide remote repositories for Git projects."

Why Do We Use Git?

- Track code changes
- Collaborate with multiple developers
- Maintain version history
- Create branches for new features
- Merge code changes safely
- Roll back to previous versions if needed

How Git Works

```
Developer
     |
git add
     |
git commit
     |
Local Git Repository
     |
```

```
git push
     |
Remote Repository (GitHub/GitLab/Bitbucket)
```

Common Git Commands

Command	Purpose
`git init`	Initialize a new Git repository
`git clone <repo-url>`	Copy a remote repository to your local machine
`git status`	Check the status of files
`git add <file>`	Stage changes
`git commit -m "message"`	Save staged changes locally
`git push`	Upload commits to the remote repository
`git pull`	Download and merge changes from the remote repository
`git fetch`	Download changes without merging
`git branch`	List or create branches
`git checkout <branch>`	Switch branches
`git merge <branch>`	Merge one branch into another
`git log`	View commit history

Git Workflow

```
Working Directory
        |
    git add
        |
Staging Area
        |
 git commit
        |
Local Repository
        |
  git push
        |
Remote Repository
```

Real-Time DevOps Example

A developer writes new code.

1. Checks the status:

```
git status
```

2. Adds changes:

```
git add .
```

3. Commits changes:

```
git commit -m "Added login feature"
```

4. Pushes to the remote repository:

```
git push origin main
```

5. Jenkins detects the new commit and starts the CI/CD pipeline automatically.

Git vs GitHub

Git	GitHub
Version control tool	Cloud-based Git repository hosting service
Installed on your computer	Stores remote Git repositories
Tracks code changes	Enables collaboration, pull requests, and code reviews
Works offline	Requires internet for remote operations

Common Interview Questions

Q1: What is the difference between `git pull` and `git fetch`?

Answer:

- `git fetch` downloads changes from the remote repository but **does not merge** them.
- `git pull` downloads **and automatically merges** the changes into the current branch.

Q2: What is a branch?

Answer:

A branch is an independent line of development that allows developers to work on new features or bug fixes without affecting the main codebase.

Q3: What is a commit?

Answer:

A commit is a saved snapshot of changes in the local Git repository, along with a descriptive commit message.

Q4: What is the difference between `git add` and `git commit`?

Answer:

- `git add` stages changes for the next commit.
- `git commit` permanently records the staged changes in the local repository.

Easy Memory Trick

Remember the Git workflow:

Edit → Add → Commit → Push

- **Edit** → Modify files
- **Add** → Stage changes
- **Commit** → Save changes locally
- **Push** → Upload changes to the remote repository

2-Minute Interview Summary

"Git is a distributed version control system used to track changes in source code and enable collaboration among developers. It provides features such as branching, merging, version history, and rollback. In DevOps, Git is used to store application code, infrastructure code, and pipeline configurations. A typical workflow is to modify files, stage them with `git add`, save them using `git commit`, and upload them to a remote repository

with `git push`. Git integrates with CI/CD tools such as Jenkins to automate application builds and deployments."

☆ Interview Tip

A common follow-up question is:

Q: How have you used Git in your projects?

A good answer is:

"I use Git for version control by cloning repositories, creating branches for development, committing code changes with meaningful messages, pushing changes to the remote repository, and pulling the latest updates before starting work. In a CI/CD environment, Jenkins automatically triggers a build whenever new code is pushed to the repository."

What is Jenkins? (Interview Answer)

Definition

Jenkins is an **open-source automation server** used to automate the **Continuous Integration (CI)** and **Continuous Delivery/Deployment (CD)** process. It automatically builds, tests, and deploys applications whenever code changes are pushed to a Git repository.

Interview Answer (2 Minutes)

"Jenkins is an open-source CI/CD automation tool used to automate software development tasks such as building, testing, and deploying applications. It integrates with Git to detect code changes, uses build tools like Maven or Gradle to compile the application, runs automated tests, and deploys the application to servers or cloud platforms. Jenkins helps reduce manual effort, improves software quality, and enables faster and more reliable releases."

Why Do We Use Jenkins?

- Automates repetitive tasks
- Builds applications automatically
- Runs automated tests
- Deploys applications
- Integrates with Git, Docker, Kubernetes, and AWS
- Reduces manual errors
- Speeds up software delivery

Jenkins CI/CD Workflow

```
Developer
     |
git push
     |
Git Repository
     |
Webhook/Poll SCM
     |
Jenkins
     |
Build (Maven/Gradle)
     |
Run Tests
     |
Create Artifact (.jar/.war)
     |
Deploy to Server (EC2/Tomcat/Docker)
```

Jenkins Architecture

```
Developer
     |
GitHub/GitLab
     |
Jenkins Controller
     |
Jenkins Agent(s)
     |
Build & Test
     |
Deploy
```

- **Controller (Master)**: Manages jobs and schedules builds.
- **Agent (Node)**: Executes build and test jobs.

Jenkins Pipeline Stages

A typical pipeline includes:

1. **Checkout** – Download code from Git.
2. **Build** – Compile the application.
3. **Test** – Run automated tests.
4. **Package** – Create JAR/WAR or Docker image.
5. **Deploy** – Deploy to the target environment.

Real-Time DevOps Example

Suppose a Java developer pushes code to Git.

1. Developer runs:

```
git push origin main
```

2. Jenkins detects the change.
3. Jenkins:
    - Downloads the latest code.
    - Runs Maven:

```
mvn clean package
```

4. Creates a `.jar` file.
5. Deploys the application to an EC2 instance or Tomcat server.
6. Sends a success or failure notification.

Important Jenkins Components

Component	Purpose
Job	Executes a specific task
Pipeline	Defines the CI/CD workflow
Agent	Executes build jobs
Plugin	Adds integrations (Git, Docker, AWS, etc.)
Workspace	Directory where Jenkins stores project files during a build

---

Common Interview Questions

Q1: What is Continuous Integration (CI)?

Answer:

Continuous Integration is the practice of automatically building and testing code whenever developers commit changes to a shared repository.

Q2: What is Continuous Delivery (CD)?

Answer:

Continuous Delivery automates the release process so that software is always ready for deployment. Deployment to production may still require manual approval.

Q3: What is the difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery	Continuous Deployment
Requires manual approval before production	Automatically deploys to production after tests pass

---

Q4: How does Jenkins know that new code has been pushed?

Answer:

- Using a **Git Webhook** (preferred)
- Or by **polling the SCM repository** at regular intervals

Q5: What build tools can Jenkins use?

Answer:

- Maven
- Gradle
- Ant

- npm (for Node.js)
- Make (for C/C++)

Easy Memory Trick

Remember:

Git → Jenkins → Build → Test → Deploy

- **Git** → Source code
- **Jenkins** → Automation
- **Build** → Compile application
- **Test** → Verify code
- **Deploy** → Release application

2-Minute Interview Summary

"Jenkins is an open-source CI/CD automation server that automates building, testing, and deploying applications. It integrates with Git repositories to detect code changes, uses tools like Maven or Gradle to build applications, executes automated tests, and deploys artifacts to target environments such as EC2, Tomcat, Docker, or Kubernetes. Jenkins supports pipelines, plugins, distributed builds through agents, and helps teams deliver software faster, more consistently, and with fewer manual errors."

☆ Interview Tip

If the interviewer asks:

"What Jenkins jobs have you created?"

A good answer is:

"I have created Pipeline jobs that pull source code from Git, build Java applications using Maven, archive the generated JAR/WAR artifact, and deploy it to an EC2 instance. I also configured build triggers using Git webhooks and integrated email notifications so the team is informed about build success or failure."

This demonstrates practical knowledge of a typical CI/CD workflow.

What is Docker? (Interview Answer)

Definition

Docker is an **open-source containerization platform** used to package an application along with its dependencies, libraries, and configuration into a **container**. This ensures the application runs consistently across development, testing, and production environments.

Interview Answer (2 Minutes)

"Docker is an open-source containerization platform that packages an application and all its dependencies into a lightweight container. Containers share the host operating system kernel, making them faster and more efficient than virtual machines. Docker helps developers and operations teams build, ship, and run applications consistently across different environments. In DevOps, Docker is widely used for application deployment, microservices, and CI/CD pipelines."

Why Do We Use Docker?

- Consistent application deployment
- Eliminates the "it works on my machine" problem
- Faster startup than Virtual Machines
- Lightweight and portable
- Easy to scale
- Ideal for microservices and CI/CD

Docker Architecture

```
Developer
     |
Dockerfile
     |
docker build
     |
Docker Image
     |
docker run
     |
```

Docker Components

Component	Purpose
Docker Engine	Runs Docker containers
Docker Image	Read-only template used to create containers
Docker Container	Running instance of an image
Dockerfile	File containing instructions to build an image
Docker Hub	Public repository for Docker images
Docker Registry	Stores Docker images (public or private)

Docker Workflow

```
Application Code
       |
Dockerfile
       |
docker build
       |
Docker Image
       |
docker run
       |
Container
```

Docker vs Virtual Machine

Docker	Virtual Machine
Uses host OS kernel	Has its own guest OS
Lightweight	Heavier
Starts in seconds	Takes minutes to boot
Low resource usage	Higher CPU and memory usage
Best for microservices	Best for complete OS isolation

Common Docker Commands

Check Docker Version

```
docker --version
```

Download an Image

```
docker pull nginx
```

List Images

```
docker images
```

Run a Container

```
docker run -d -p 80:80 nginx
```

List Running Containers

```
docker ps
```

List All Containers

```
docker ps -a
```

Stop a Container

```
docker stop <container_id>
```

Remove a Container

```
docker rm <container_id>
```

Remove an Image

```
docker rmi <image_id>
```

Real-Time Example

Suppose you have a Java Spring Boot application.

1. Build the application:

```
mvn clean package
```

2. Build the Docker image:

```
docker build -t springboot-app .
```

3. Run the container:

```
docker run -d -p 8080:8080 springboot-app
```

Users can now access the application on port **8080**.

Dockerfile Example

```
FROM openjdk:17-jdk
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

This Dockerfile:

- Uses Java 17
- Copies the application JAR
- Exposes port 8080
- Starts the application

Common Interview Questions

Q1: What is the difference between an Image and a Container?

Answer:

- **Image:** A read-only template used to create containers.
- **Container:** A running instance of an image.

Q2: What is Docker Hub?

Answer:

Docker Hub is a cloud-based repository where Docker images are stored and shared.

Q3: Why is Docker faster than a Virtual Machine?

Answer:

Docker containers share the host operating system kernel, so they don't need to boot a separate operating system, making them much faster and more lightweight.

Q4: What is a Dockerfile?

Answer:

A Dockerfile is a text file containing instructions used to build a Docker image automatically.

Easy Memory Trick

Remember:

Code → Image → Container

- **Code** → Application + Dockerfile
- **Image** → Built package
- **Container** → Running application

2-Minute Interview Summary

"Docker is an open-source containerization platform that packages applications and their dependencies into lightweight containers. Containers share the host operating system kernel, making them more efficient than virtual machines. Docker consists of images, containers, Dockerfiles, and registries such as Docker Hub. In DevOps, Docker is used to build, package, and deploy applications consistently across environments and integrates seamlessly with CI/CD tools like Jenkins."

☆ Interview Tip

If the interviewer asks:

"How have you used Docker in your project?"

A good answer is:

"I containerized a Java Spring Boot application by creating a Dockerfile, building the Docker image using `docker build`, and running it as a container using `docker run`. In the CI/CD pipeline, Jenkins built the application, created the Docker image, and deployed it to the target environment. This ensured consistent deployments across development, testing, and production.

What is Terraform? (Interview Answer)

Definition

Terraform is an **Infrastructure as Code (IaC)** tool developed by **HashiCorp**. It allows you to create, update, and manage cloud infrastructure using configuration files instead of manually creating resources through the AWS Management Console.

Interview Answer (2 Minutes)

"Terraform is an open-source Infrastructure as Code tool used to automate the provisioning and management of cloud resources. It uses HashiCorp Configuration Language (HCL) to define infrastructure such as VPCs, EC2 instances, S3 buckets, and RDS databases in code. Terraform enables consistent, repeatable deployments, supports version control through Git, and can manage infrastructure across multiple cloud providers including AWS, Azure, and Google Cloud."

Why Do We Use Terraform?

- Automates infrastructure provisioning
- Eliminates manual configuration
- Ensures consistent environments
- Supports version control with Git
- Enables reusable infrastructure through modules
- Works across multiple cloud providers

Terraform Workflow

```
Write Terraform Code (.tf)
          |
terraform init
          |
terraform plan
          |
terraform apply
          |
AWS Infrastructure Created
```

Terraform Architecture

```
Terraform Code
      |
Terraform CLI
      |
AWS Provider
      |
AWS API
      |
EC2 | VPC | S3 | RDS | IAM
```

Terraform Commands

Command	Purpose
`terraform init`	Initialize the working directory and download providers
`terraform validate`	Validate the configuration syntax
`terraform fmt`	Format Terraform files
`terraform plan`	Preview the changes Terraform will make
`terraform apply`	Create or update infrastructure
`terraform destroy`	Delete the infrastructure
`terraform show`	Display the current state
`terraform output`	Show output values

Terraform Lifecycle

Step 1: Write Code

Example:

```
resource "aws_instance" "web" {
  ami           = "ami-xxxxxxxx"
```

```
  instance_type = "t2.micro"
}
```

Step 2: Initialize

```
terraform init
```

Downloads the required AWS provider.

Step 3: Preview Changes

```
terraform plan
```

Shows:

- Resources to create
- Resources to modify
- Resources to destroy

No changes are made yet.

Step 4: Apply Changes

```
terraform apply
```

Creates the infrastructure in AWS.

Step 5: Destroy (If Required)

```
terraform destroy
```

Deletes all resources managed by Terraform.

Real-Time Example

Suppose you need to create:

- VPC
- Public and Private Subnets
- Internet Gateway
- EC2 Instance
- Security Group

Instead of creating each resource manually in the AWS Console, you define them in Terraform files and deploy everything with:

```
terraform apply
```

Terraform provisions the complete infrastructure automatically.

Terraform State File

Terraform maintains a **state file** (`terraform.tfstate`) that tracks the infrastructure it manages.

Purpose:

- Maps Terraform resources to actual cloud resources.
- Determines what changes are needed during future `terraform plan` or `terraform apply` operations.

In production, the state file is often stored remotely (for example, in an S3 bucket with state locking using DynamoDB).

Terraform vs CloudFormation

Terraform	AWS CloudFormation
Multi-cloud (AWS, Azure, GCP, etc.)	AWS only
Uses HCL	Uses JSON or YAML
Open source	AWS managed service
Large module ecosystem	Native AWS integration

Common Interview Questions

Q1: What is Infrastructure as Code (IaC)?

Answer:

Infrastructure as Code is the practice of defining and managing infrastructure using code instead of manual configuration, making deployments consistent, repeatable, and version-controlled.

Q2: What is `terraform plan`?

Answer:

`terraform plan` previews the changes Terraform will make without modifying the infrastructure.

Q3: What is `terraform apply`?

Answer:

`terraform apply` creates or updates infrastructure based on the Terraform configuration.

Q4: What is the Terraform state file?

Answer:

The Terraform state file records the current infrastructure managed by Terraform and helps determine what changes are needed in future deployments.

Easy Memory Trick

Remember:

Write → Init → Plan → Apply

- **Write** → Create `.tf` files
- **Init** → Download providers
- **Plan** → Preview changes
- **Apply** → Create infrastructure

2-Minute Interview Summary

"Terraform is an open-source Infrastructure as Code tool used to automate cloud infrastructure provisioning. It uses HCL to define resources such as EC2 instances, VPCs, S3 buckets, and RDS databases. The typical workflow is `terraform init` to initialize the project, `terraform plan` to preview changes, and `terraform apply` to provision infrastructure. Terraform stores infrastructure information in a state file and supports reusable modules and version control through Git. It is widely used in DevOps to automate infrastructure deployments consistently and efficiently."

☆ Interview Tip

If the interviewer asks:

"How have you used Terraform in your project?"

A good answer is:

"I used Terraform to provision AWS infrastructure, including VPCs, subnets, Security Groups, EC2 instances, and S3 buckets. The Terraform code was stored in Git, and Jenkins executed `terraform plan` and `terraform apply` as part of the deployment pipeline. This ensured consistent, repeatable, and automated infrastructure provisioning across environments."

Note: Only say you have used Terraform in a project if that is true. If you have only practiced it in a lab, say:

"I have hands-on experience creating AWS resources such as EC2 instances, VPCs, and S3 buckets using Terraform in a lab environment, and I understand the complete workflow from `terraform init` to `terraform apply`."

This is an honest answer and is appropriate for interview situations.